



Introducción y Configuración del Entorno

¿Qué es un *pipeline* de datos?

Los *pipelines* de datos son conjuntos de procesos que trasladan y transforman datos desde diversas fuentes hasta un destino donde se puede generar nuevo valor. Son la base del análisis, generación de informes y aprendizaje automático (Densmore, 2021).

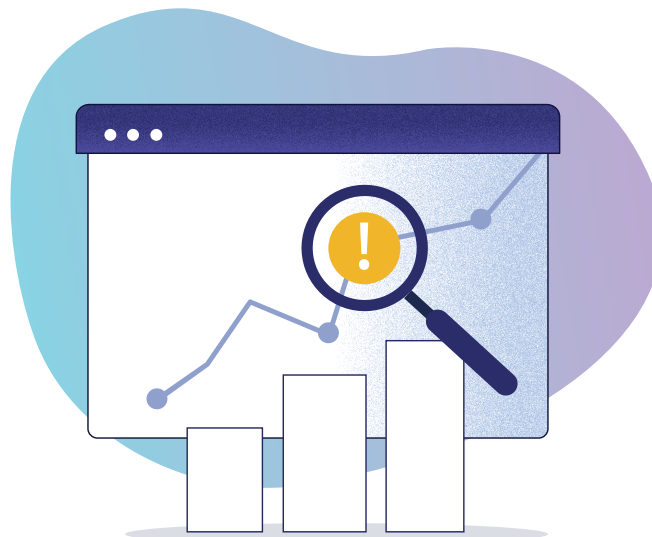
De manera general, esta secuencia consiste en los siguientes pasos:

1. Extracción de datos (*Extraction*)

Primero, necesitas obtener los datos desde diferentes fuentes, por ejemplo:

- **Base de datos transaccional:** extraes información detallada de las ventas realizadas en las últimas 24 horas mediante una consulta SQL.
- **API de inventario:** recuperas datos del inventario actual para relacionarlos con los productos vendidos a través de llamadas a la API.
- **Archivos de bitácora (*logs*) de la web:** descargas bitácoras de actividad del sitio web para analizar el comportamiento de navegación de los usuarios antes de realizar una compra.

Cada una de estas fuentes tiene su propio formato, protocolo de acceso y esquema de datos. La extracción debe coordinarse para asegurar que todos los datos correspondan al mismo periodo de tiempo.

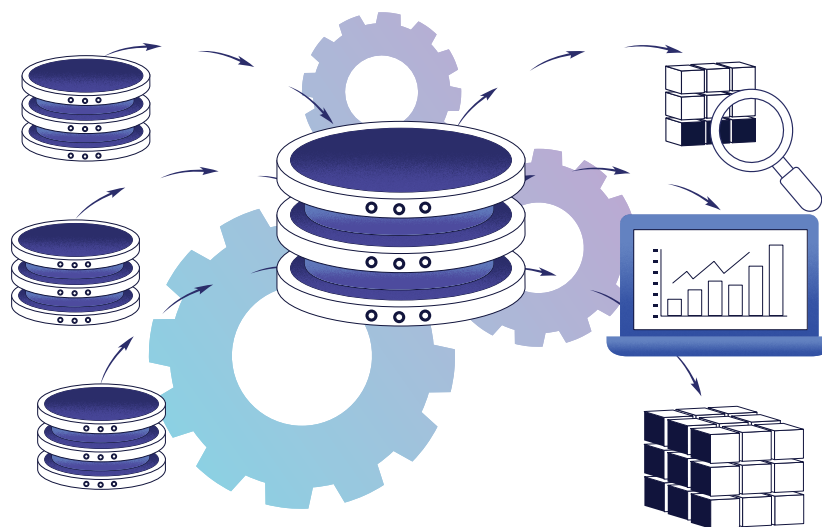


2. Limpieza y validación (*Cleaning*)

Una vez extraídos, los datos pueden requerir preprocesamiento:

- **Validación de esquema:** verificas que los campos críticos (identificadores de producto, precios, cantidades, fechas, etc.) estén presentes y tengan el formato correcto.
- **Eliminación de duplicados:** identificas y eliminas transacciones duplicadas que pueden ocurrir por problemas en el sistema de ventas.
- **Manejo de valores nulos:** aplicas reglas para tratar datos faltantes, como precios o categorías de productos no definidas.
- **Corrección de anomalías:** detectas y corriges valores imposibles (como cantidades negativas o precios excesivamente altos).

En esta etapa, generas bitácoras de control para documentar cuántos registros fueron rechazados y por qué motivos.



3. Transformación (*Transformation*)

Ahora, transformas los datos limpios en información relevante para el negocio:

- **Normalización:** estandarizas los nombres de categorías de productos que pueden variar entre sistemas.
- **Agregación:** calculas métricas por periodos de tiempo, por categoría de producto, por región geográfica, por canal de ventas, etc.
- **Enriquecimiento:** añades datos adicionales como margen de beneficio por producto, costo de adquisición del cliente, etc.
- **Creación de características:** generas variables derivadas como “días desde el último reabastecimiento” o “tasa de conversión por visita”.

Esta fase puede implicar la aplicación de lógica de negocio compleja y puede requerir conjuntar múltiples conjuntos de datos.



4. Carga (Loading)

Finalmente, cargas los datos procesados en sus destinos:

- **Data Warehouse:** almacenas las tablas agregadas en tu almacén de datos analítico para uso por los equipos de análisis.
- **Sistema de BI:** actualizas las tablas específicas que alimentan los sistemas y tableros (*dashboards*) de inteligencia de negocios.
- **Sistemas de aprendizaje:** envías los datos más recientes al servicio que ejecuta tus modelos de aprendizaje automático, por ejemplo, predicción de demanda, *clustering* de clientes, etc.
- **Almacenamiento de archivos:** guardas copias de los datos procesados en formatos estandarizados en un *data lake* para archivado y análisis posterior.

Cada destino puede requerir formatos específicos y procesos de carga optimizados.

5. Orquestación y monitoreo

Para completar el *pipeline*, necesitas:

- **Programación y calendarización:** configuras el *pipeline* para que se ejecute automáticamente cada domingo por la mañana a las 4:00 AM. O bien, puedes programar el *pipeline* para que esté procesando los datos que se insertan en sus fuentes de origen.
- **Monitoreo:** implementas alertas que te notifican si el *pipeline* falla o si hay anomalías en los datos (como un descenso repentino en el volumen de ventas).
- **Registro:** mantienes bitácoras detalladas de cada ejecución para auditoría y solución de problemas.

Cada paso depende de los anteriores:

1. No puedes transformar datos que no has extraído primero.
2. No es seguro cargar datos que no has limpiado y validado.
3. No puedes generar informes significativos sin haber aplicado las transformaciones de negocio adecuadas.

Si cualquier paso falla (por ejemplo, si la API de inventario no está disponible), todo el *pipeline* puede verse comprometido, lo que afecta a los informes y *dashboards* que utilizan esos datos para la toma de decisiones.

Por otro lado, posiblemente estás pensando: “esto se parece mucho a los procesos ETL (*Extract-Transform-Load*, por sus siglas en inglés) de hace unos años”, y tienes razón.

Los procesos que extraen, transforman y cargan datos han existido por décadas; sin embargo, un *pipeline* de datos es un **concepto más general** que describe cualquier flujo automatizado de datos a través de una serie de procesos o transformaciones, independientemente de su propósito, arquitectura, origen y destino. Esto incluye a los ETL tradicionales, que en la actualidad se consideran un patrón más de los *pipelines* (Densmore, 2021).

Recuerda que los ETL alimentaban normalmente almacenes de datos (*data warehouses*). Un *pipeline* puede tomar **múltiples fuentes de datos** y procesar y enviar esos datos a **múltiples destinos**, desde un *data warehouse* hasta un sistema de aprendizaje automático.

Introducción a Kedro y sus ventajas

Existen múltiples herramientas, metodologías y marcos de trabajo (*frameworks*) que te permiten implementar canalizaciones de datos en cualquier ambiente, desde instalaciones locales hasta la nube.

Cada una de estas herramientas tiene sus propias metodologías, filosofías y ventajas, así que es importante conocer una variedad de herramientas para poder seleccionar la más adecuada para cada circunstancia.

Dentro de esta gama de múltiples herramientas está Kedro, un *framework* de código abierto en Python que implementa buenas prácticas de ingeniería de *software* para crear *pipelines* de datos y código de ciencia de datos mantenibles, reproducibles y modulares (Kedro, n.d.).

Desarrollado por QuantumBlack (McKinsey & Company, 2019), actualmente mantenido por la comunidad bajo la Linux Foundation AI & Data (LF AI & DATA, n.d.), Kedro proporciona una estructura estandarizada para proyectos de *pipelines* de datos que facilita la colaboración y el despliegue.

El núcleo de Kedro se basa en la organización de un proyecto de datos en tres componentes principales:

1. **Nodos:** funciones de Python puras que realizan una tarea específica, como cargar, transformar o modelar datos.
2. **Pipelines:** secuencias de nodos conectados donde las salidas de un nodo pueden servir como entradas para otros.
3. **Catálogo de datos:** registro centralizado de todos los conjuntos de datos que utiliza el proyecto, definiendo cómo deben cargarse y guardarse.

Ventajas de Kedro

1. Estructura de proyecto estandarizada

Kedro proporciona una estructura de directorios y archivos predefinida:

- Código fuente
- Datos
- Configuraciones
- Documentación
- Pruebas

Esta estructura facilita que cualquier persona familiarizada con Kedro pueda entender rápidamente la organización de un proyecto, independientemente de su complejidad.

2. Separación de conceptos (*Concerns*)

Kedro implementa una clara separación entre:

- La lógica del procesamiento de datos (nodos y *pipelines*).
- La configuración del acceso a los datos (*Data Catalog*).
- Los parámetros del proyecto.

Esta separación hace que el **código** sea **modular y fácil de probar y de mantener**.

3. Reproducibilidad

Al definir explícitamente los *pipelines* y gestionar automáticamente las dependencias entre nodos, Kedro garantiza que:

- Los flujos de trabajo sean reproducibles.
- El seguimiento del linaje de datos sea posible.

4. Modularidad y reutilización

Kedro fomenta la creación de:

- Funciones pequeñas y bien definidas (nodos).
- *Pipelines* independientes que pueden ejecutarse individualmente.
- Componentes que pueden reutilizarse en diferentes proyectos.

5. Flexibilidad y extensibilidad

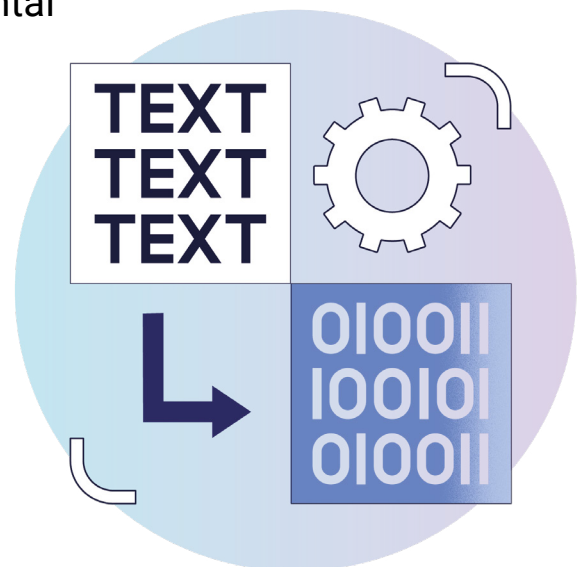
Kedro es altamente extensible a través de:

- **Hooks** para personalizar el comportamiento.
- **Plugins** para añadir funcionalidades específicas.
- Soporte para múltiples tipos de datos y formatos.
- Integración con diferentes sistemas de orquestación.

6. Transición de desarrollo a producción

Kedro facilita la transición de código experimental a producción mediante:

- Empaquetado de proyectos.
- Integración con herramientas de orquestación como Airflow, Prefect, Databricks, etc.
- Despliegue en diferentes entornos (local, nube, etc.).



7. Visualización y documentación

Kedro ofrece:

- Kedro-Viz para visualizar *pipelines*.
- Documentación generada automáticamente.
- Seguimiento de experimentos y resultados para *pipelines* de aprendizaje.

8. Colaboración mejorada

La estructura estandarizada y las buenas prácticas de Kedro facilitan:

- El trabajo en equipo.
- El control de versiones del código.
- La revisión de código.
- La transferencia de conocimiento.

Cierre

En resumen, Kedro:

- Se centra en la estructura del código y las buenas prácticas de ingeniería de software.
- No impone un sistema de orquestación específico.
- Es agnóstico en cuanto a las herramientas de modelado o visualización.
- Se integra bien con el ecosistema existente de Python para datos.
- Al estar basado en Python, es multiplataforma de manera nativa, permitiendo implementar *pipelines* en Linux, Windows o macOS.

Estas características convierten a Kedro en una herramienta ideal tanto para científicos de datos que buscan estructurar mejor su trabajo como para ingenieros de datos que necesitan implementar flujos de datos robustos y mantenibles.

Material adicional

Chocolatey Software. (s.f.). *The Package Manager for Windows* [El gestor de paquetes para Windows]. Chocolatey. Recuperado el 28 de marzo de 2025, de <https://chocolatey.org/>

Homebrew. (s.f.). *Install Homebrew* [Instala Homebrew]. Homebrew. Recuperado el 28 de marzo de 2025, de <https://brew.sh/>

Kedro. (s.f.). *A toolbox for production-ready data science* [A toolbox for production-ready data science]. Kedro. Recuperado el 28 de marzo de 2025, de: <https://docs.kedro.org/en/stable/index.html>

Python Packaging Authority. (s.f.). *Install packages in a virtual environment using pip and venv* [Instalación y uso de pip y entornos virtuales]. Python. Recuperado el 28 de marzo de 2025 de: <https://packaging.python.org/en/latest/guides/installing-using-pip-and-virtual-environments/>

Python Software Foundation. (s.f.). *venv – Creation of virtual environments* [venv- Creación de entornos virtuales]. Python. Recuperado el 28 de marzo de 2025 de: <https://docs.python.org/3/library/venv.html>

Python Software Foundation. (s.f.). *Python Releases for Windows* [Lanzamientos de Python para Windows]. Python. Recuperado el 28 de marzo de 2025, de <https://www.python.org/downloads/windows/>

Python Software Foundation. (s.f.). *Python Releases for MacOS* [Lanzamientos de Python para MacOS]. Python. Recuperado el 28 de marzo de 2025, de <https://www.python.org/downloads/macos/>

Real Python. (s.f.). *Installing Python* [Instalación de Python]. Real Python. Recuperado el 28 de marzo de 2025 de: <https://realpython.com/installing-python/>

Visual Studio Code. (s.f.). *Download Visual Studio Code* [Descargar Visual Studio Code]. Visual Studio Code. Recuperado el 28 de marzo de 2025, de <https://code.visualstudio.com/Download>

Visual Studio Code. (s.f.). *Tutorial: Get started with Visual Studio Code* [Primeros pasos con Visual Studio Code]. Visual Studio Code. Recuperado el 28 de marzo de 2025 de: <https://code.visualstudio.com/docs/getstarted/getting-started>

Visual Studio Code. (s.f.). *Visual Studio Code on Linux* [Configuración de Visual Studio Code en Linux]. Visual Studio Code. Recuperado el 28 de marzo de 2025, de <https://code.visualstudio.com/docs/setup/linux>

Nota: Los enlaces a las páginas web incluidos en este material adicional fueron consultados y verificados por última vez el 24 de noviembre de 2025 y su disponibilidad depende del autor.

Bibliografía

Densmore, J. (2021). *Data Pipelines Pocket Reference*. O'Reilly Media, Inc.

Google. (s.f.). *What is a data lake?* [¿Qué es un data lake?] Google Cloud. Recuperado el 28 de marzo de 2025, de: <https://cloud.google.com/learn/what-is-a-data-lake?hl=es>

Kedro. (s.f.). *Learn Kedro with hands-on video* [Aprende Kedro con un video práctico]. Kedro. Recuperado el 28 de marzo de 2025, de: <https://docs.kedro.org/en/stable/introduction/index.html>

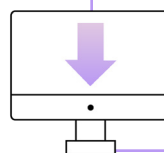
Kedro. (s.f.) *Set up Kedro* [Configurar Kedro]. Kedro. Recuperado el 28 de marzo de 2025, de: https://docs.kedro.org/en/stable/get_started/install.html

LF AI & Data Foundation. (s.f.). *Kedro*. LF AI & Data. Recuperado el 28 de marzo de 2025, de: <https://lfaidata.foundation/projects/kedro/>

McKinsey & Company. (6 de junio de 2019). *Meet Kedro, McKinsey's first open-source software tool* [Conoce Kedro, la primera herramienta de software de código abierto de McKinsey] McKinsey & Company. Recuperado el 28 de marzo de 2025, de: <https://www.mckinsey.com/about-us/new-at-mckinsey-blog/meet-kedro-mckinseys-first-open-source-software-tool>

The Python Package Index. (s.f) *kedro 0.19.11*. The Python Package Index. Recuperado el 28 de marzo de 2025, de: <https://pypi.org/project/kedro/>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 24 de noviembre de 2025 y su disponibilidad depende del autor.



Descarga este documento en la sección “**Archivos adjuntos**” de la plataforma.